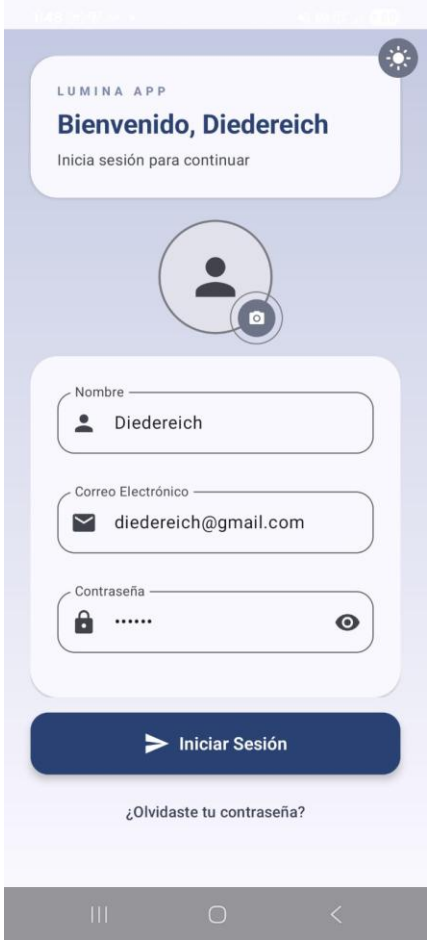
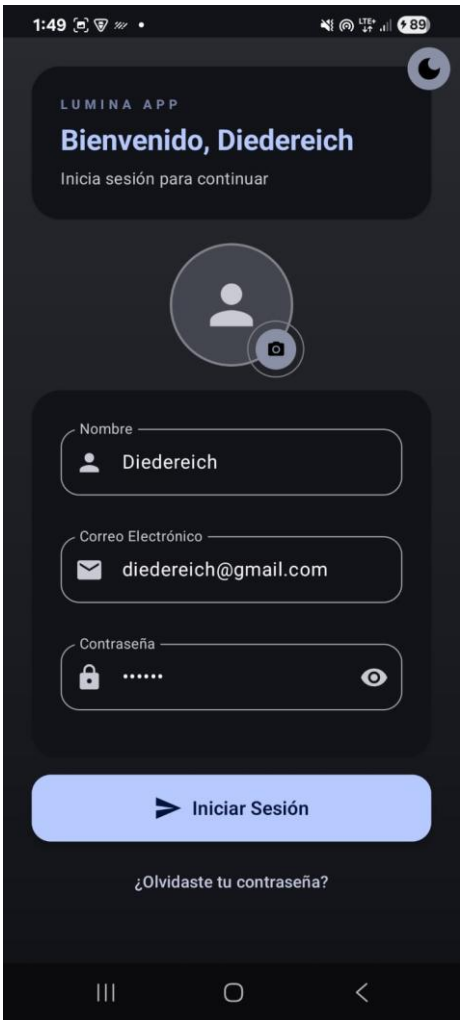
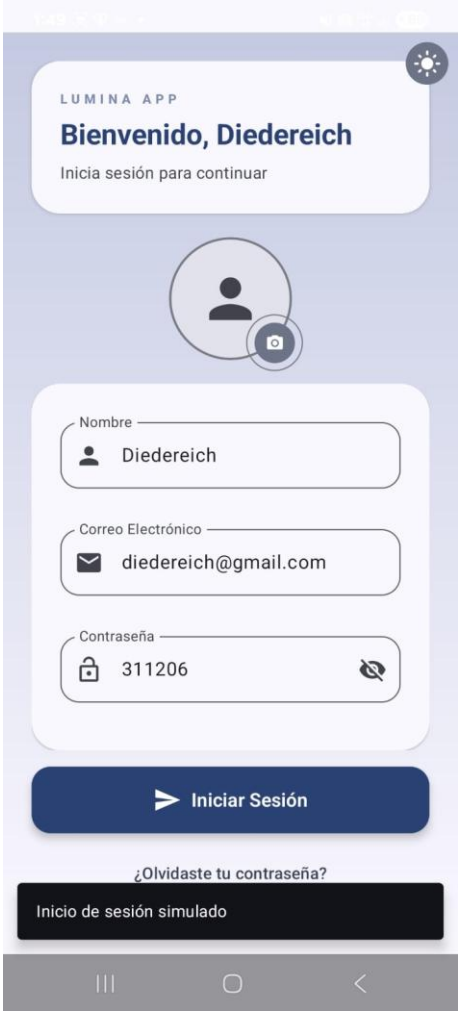
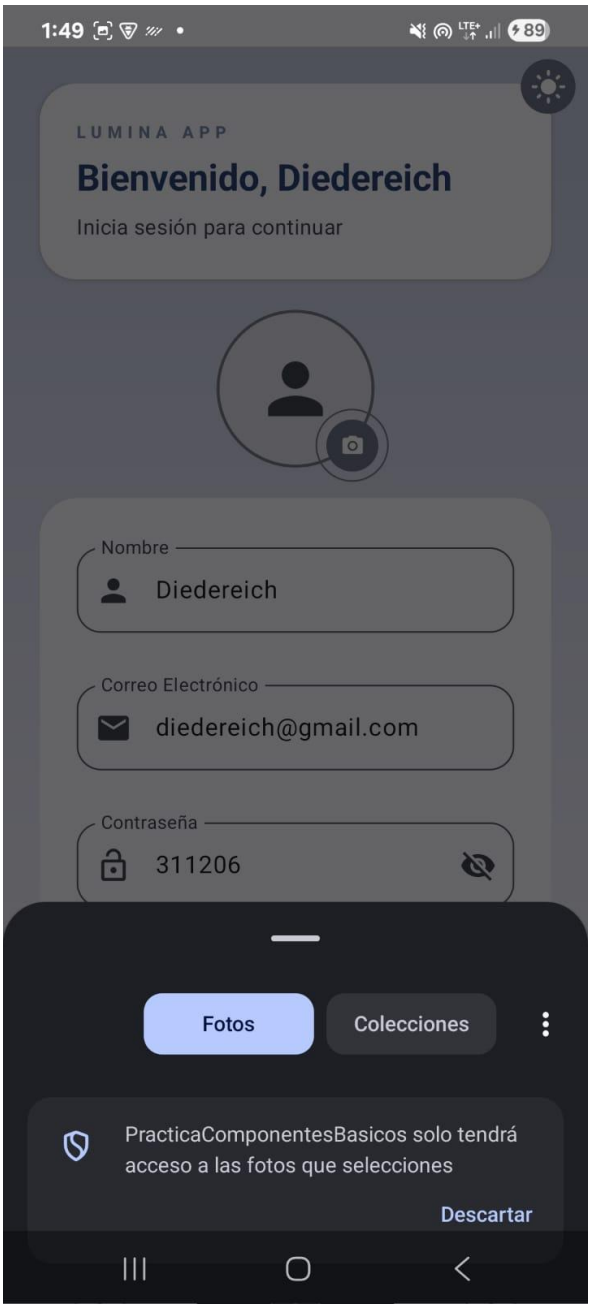
La aplicación es una pantalla de inicio de sesión en Jetpack Compose (LoginScreen.kt) con enfoque visual moderno y validación en tiempo real. Permite ingresar nombre, correo y contraseña, seleccionar/cambiar foto de perfil con un selector minimalista, alternar tema claro/oscuro y ejecutar un inicio de sesión simulado con retroalimentación mediante Snackbar. Además, el encabezado se personaliza dinámicamente con “Bienvenido, {Nombre}”, manteniendo una presentación limpia incluso con nombres largos.

Tras la validación, se reforzaron los detalles de experiencia de usuario: animación en el botón de iniciar sesión con ícono de avión de papel desplazándose de izquierda a derecha durante la carga, mejora del campo de contraseña con ícono de candado abierto/cerrado animado según visibilidad, capitalización automática de la primera letra del nombre, eliminación del campo apellido, reducción de espacios excesivos entre cajas, reposicionamiento minimalista del botón de tema y ajuste del comportamiento del teclado (KeyboardOptions/ImeAction) para adaptarse mejor al campo activo y hacer el flujo más fluido.




1:49
LUMINA APP
Bienvenido, Diedereich
Inicia sesión para continuar
Nombre
Diedereich
Correo Electrónico
diedereich@gmail.com
Contraseña
Iniciar Sesión
¿Olvidaste tu contraseña?


LUMINA APP
Bienvenido, Diedereich
Inicia sesión para continuar
Nombre
Diedereich
Correo Electrónico
diedereich@gmail.com
Contraseña
311206
Iniciar Sesión
¿Olvidaste tu contraseña?
Inicio de sesión simulado


1:49
89
LUMINA APP
Bienvenido, Diedereich
Inicia sesión para continuar
Nombre
Diedereich
Correo Electrónico
diedereich@gmail.com
Contraseña
311206
Fotos
Colecciones
PracticaComponentesBasicos solo tendrá acceso a las fotos que selecciones
Descartar

Código Fuente

```
package ni.edu.uam.practicacomponentesbasicos

import android.net.Uri
import android.util.Patterns
import androidx.activity.compose.rememberLauncherForActivityResult
import androidx.activity.result.PickVisualMediaRequest
import androidx.activity.result.contract.ActivityResultContracts
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.Crossfade
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.animateContentSize
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.animation.slideInVertically
import androidx.compose.animation.slideOutVertically
import androidx.compose.animation.core.FastOutSlowInEasing
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.animation.core.animateFloatAsState
import androidx.compose.animation.core.tween
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.Send
import androidx.compose.material.icons.filled.DarkMode
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.LightMode
import androidx.compose.material.icons.filled.Lock
```

```
import androidx.compose.material.icons.filled.LockOpen
import androidx.compose.material.icons.filled.Person
import androidx.compose.material.icons.filled.PhotoCamera
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.draw.scale
import androidx.compose.ui.focus.FocusDirection
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.graphicsLayer
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.layout.onSizeChanged
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.platform.LocalSoftwareKeyboardController
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardCapitalization
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.text.style.TextOverflow
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import coil.compose.AsyncImage
import kotlinx.coroutines.delay
import kotlinx.coroutines.launch
import
ni.edu.uam.practicacomponentesbasicos.ui.theme.PracticaComponentesBasic
osTheme

private val ScreenPadding = 24.dp
private val SectionSpacing = 14.dp
private val ItemSpacing = 10.dp
```

```
private val CardShape = RoundedCornerShape(24.dp)
private val FieldShape = RoundedCornerShape(16.dp)

@Composable
fun LoginScreen() {
  var isDarkTheme by rememberSaveable { mutableStateOf(false) }
  var firstName by rememberSaveable { mutableStateOf("") }
  var email by rememberSaveable { mutableStateOf("") }
  var password by rememberSaveable { mutableStateOf("") }
  var isPasswordVisible by rememberSaveable { mutableStateOf(false) }
  var isLoading by rememberSaveable { mutableStateOf(false) }
  var selectedImageUri by remember { mutableStateOf<Uri?>(null) }
  val snackbarHostState = remember { SnackbarHostState() }
  val scope = rememberCoroutineScope()

  val isFirstNameValid by remember(firstName) { derivedStateOf {
firstName.trim().length >= 2 } }
  val isEmailValid by remember(email) { derivedStateOf { isValidEmail(email) } }
  val isPasswordValid by remember(password) { derivedStateOf {
password.length >= 6 } }
  val canLogin = isFirstNameValid && isEmailValid && isPasswordValid

  val photoPickerLauncher = rememberLauncherForActivityResult(
    contract = ActivityResultContracts.PickVisualMedia(),
    onResult = { uri -> selectedImageUri = uri }
  )

  PracticaComponentesBasicosTheme(darkTheme = isDarkTheme) {
    Scaffold(
      containerColor = MaterialTheme.colorScheme.background,
      snackbarHost = { SnackbarHost(hostState = snackbarHostState) }
    ) { innerPadding ->
      LoginContent(
        modifier = Modifier.padding(innerPadding),
        isDarkTheme = isDarkTheme,
        onThemeChange = { isDarkTheme = it },
        imageUri = selectedImageUri,
        onPickImage = {
```

```
            photoPickerLauncher.launch(
PickVisualMediaRequest(ActivityResultContracts.PickVisualMedia.ImageOnly)
            )
          },
          firstName = firstName,
          onFirstNameChange = { firstName = normalizeFirstName(it) },
          email = email,
          onEmailChange = { email = it },
          password = password,
          onPasswordChange = { password = it },
          isPasswordVisible = isPasswordVisible,
          onPasswordVisibilityToggle = { isPasswordVisible = !isPasswordVisible },
          showFirstNameError = firstName.isNotEmpty() && !isFirstNameValid,
          showEmailError = email.isNotEmpty() && !isEmailValid,
          showPasswordError = password.isNotEmpty() && !isPasswordValid,
          canLogin = canLogin,
          isLoading = isLoading,
          onLoginClick = {
            if (!canLogin || isLoading) return@LoginContent
            scope.launch {
              isLoading = true
              delay(1200)
              isLoading = false
              snackbarHostState.showSnackbar("Inicio de sesión simulado")
            }
          }
        )
      }
    }
}

private fun isValidEmail(email: String): Boolean {
  return email.isNotBlank() &&
Patterns.EMAIL_ADDRESS.matcher(email).matches()
}

private fun normalizeFirstName(value: String): String {
```

```
    if (value.isEmpty()) return value
    return value.replaceFirstChar { first ->
        if (first.isLowerCase()) first.titlecase() else first.toString()
    }
}

@Composable
private fun LoginContent(
    modifier: Modifier = Modifier,
    isDarkTheme: Boolean,
    onThemeChange: (Boolean) -> Unit,
    imageUri: Uri?,
    onPickImage: () -> Unit,
    firstName: String,
    onFirstNameChange: (String) -> Unit,
    email: String,
    onEmailChange: (String) -> Unit,
    password: String,
    onPasswordChange: (String) -> Unit,
    isPasswordVisible: Boolean,
    onPasswordVisibilityToggle: () -> Unit,
    showFirstNameError: Boolean,
    showEmailError: Boolean,
    showPasswordError: Boolean,
    canLogin: Boolean,
    isLoading: Boolean,
    onLoginClick: () -> Unit
) {
    val topColor by animateColorAsState(
        targetValue = if (isDarkTheme) {
            MaterialTheme.colorScheme.surfaceContainerHigh
        } else {
            MaterialTheme.colorScheme.primaryContainer.copy(alpha = 0.35f)
        },
        animationSpec = tween(durationMillis = 450, easing = FastOutSlowInEasing),
        label = "backgroundTop"
    )
```

```
Box(
  modifier = modifier
    .fillMaxSize()
    .background(
      brush = Brush.verticalGradient(
        colors = listOf(topColor, MaterialTheme.colorScheme.background)
      )
    )
) {
  Column(
    modifier = Modifier
      .fillMaxSize()
      .padding(ScreenPadding)
      .verticalScroll(rememberScrollState()),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(SectionSpacing)
  ) {
    AnimatedVisibility(
      visible = true,
      enter = fadeIn(tween(450)) + slideInVertically(tween(450)) { it / 3 }
    ) {
      LoginHeader(firstName = firstName)
    }

    AnimatedVisibility(
      visible = true,
      enter = fadeIn(tween(550, delayMillis = 80)) +
        slideInVertically(tween(550, delayMillis = 80)) { it / 4 }
    ) {
      ProfileImageSelector(imageUri = imageUri, onPickImage = onPickImage)
    }

    AnimatedVisibility(
      visible = true,
      enter = fadeIn(tween(650, delayMillis = 140)) +
        slideInVertically(tween(650, delayMillis = 140)) { it / 4 }
    ) {
      LoginForm(
```

```
            firstName = firstName,
            onFirstNameChange = onFirstNameChange,
            email = email,
            onEmailChange = onEmailChange,
            password = password,
            onPasswordChange = onPasswordChange,
            isPasswordVisible = isPasswordVisible,
            onPasswordVisibilityToggle = onPasswordVisibilityToggle,
            showFirstNameError = showFirstNameError,
            showEmailError = showEmailError,
            showPasswordError = showPasswordError
          )
        }

        LoginFooter(
          canLogin = canLogin,
          isLoading = isLoading,
          onLoginClick = onLoginClick
        )
      }

      FilledTonalIconButton(
        onClick = { onThemeChange(!isDarkTheme) },
        modifier = Modifier
          .align(Alignment.TopEnd)
          .padding(top = 8.dp, end = 8.dp)
          .size(36.dp)
      ) {
        Crossfade(targetState = isDarkTheme, label = "themeIcon") { dark ->
          Icon(
            imageVector = if (dark) Icons.Default.DarkMode else
Icons.Default.LightMode,
            contentDescription = if (dark) "Tema oscuro activado" else "Tema claro
activado"
          )
        }
      }
    }
```

```
}

@Composable
fun LoginHeader(
  firstName: String
) {
  val trimmedName = firstName.trim()
  val welcomeText = if (trimmedName.isEmpty()) {
    "Bienvenido"
  } else {
    "Bienvenido, $trimmedName"
  }

  ElevatedCard(
    modifier = Modifier.fillMaxWidth(),
    shape = CardShape,
    colors = CardDefaults.elevatedCardColors(containerColor =
MaterialTheme.colorScheme.surface),
    elevation = CardDefaults.elevatedCardElevation(defaultElevation = 2.dp)
  ) {
    Column(
      modifier = Modifier
        .fillMaxWidth()
        .padding(ScreenPadding),
      verticalArrangement = Arrangement.spacedBy(6.dp)
    ) {
      Text(
        text = "LUMINA APP",
        style = MaterialTheme.typography.labelSmall,
        fontWeight = FontWeight.ExtraBold,
        letterSpacing = 4.sp,
        color = MaterialTheme.colorScheme.primary.copy(alpha = 0.6f)
      )

      Text(
        text = welcomeText,
        style = MaterialTheme.typography.headlineSmall,
        fontWeight = FontWeight.Bold,
```

```
            color = MaterialTheme.colorScheme.primary,
            maxLines = 1,
            overflow = TextOverflow.Ellipsis,
            modifier = Modifier.fillMaxWidth()
        )

        Text(
            text = "Inicia sesión para continuar",
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
  }
}

@Composable
fun ProfileImageSelector(
  imageUri: Uri?,
  onPickImage: () -> Unit
) {
  val borderColor by animateColorAsState(
    targetValue = if (imageUri != null) {
      MaterialTheme.colorScheme.primary
    } else {
      MaterialTheme.colorScheme.outlineVariant
    },
    animationSpec = tween(400),
    label = "avatarBorderColor"
  )
  val borderWidth by animateDpAsState(
    targetValue = if (imageUri != null) 3.dp else 2.dp,
    animationSpec = tween(400),
    label = "avatarBorderWidth"
  )
  val avatarScale by animateFloatAsState(
    targetValue = if (imageUri != null) 1f else 0.96f,
    animationSpec = tween(400),
    label = "avatarScale"
```

```
)

val avatarSize = 108.dp

Box(
  modifier = Modifier
    .size(avatarSize + 8.dp)
    .padding(4.dp),
  contentAlignment = Alignment.Center
) {
  Box(
    modifier = Modifier
      .size(avatarSize)
      .scale(avatarScale)
      .clip(CircleShape)
      .background(MaterialTheme.colorScheme.surfaceVariant)
      .border(borderWidth, borderColor, CircleShape),
    contentAlignment = Alignment.Center
  ) {
    Crossfade(targetState = imageUri, label = "avatarTransition") { uri ->
      if (uri != null) {
        AsyncImage(
          model = uri,
          contentDescription = "Imagen de perfil",
          modifier = Modifier.fillMaxSize(),
          contentScale = ContentScale.Crop
        )
      } else {
        Icon(
          imageVector = Icons.Default.Person,
          contentDescription = null,
          modifier = Modifier.size(56.dp),
          tint = MaterialTheme.colorScheme.onSurfaceVariant
        )
      }
    }
  }
```

```
      FilledTonalIconButton(
        onClick = onPickImage,
        modifier = Modifier
          .align(Alignment.BottomEnd)
          .size(34.dp)
          .border(1.dp, MaterialTheme.colorScheme.outlineVariant, CircleShape)
      ) {
        Icon(
          imageVector = Icons.Default.PhotoCamera,
          contentDescription = if (imageUri == null) "Agregar foto" else "Cambiar
foto",
          modifier = Modifier.size(16.dp)
        )
      }
    }
  }

@Composable
fun LoginForm(
  firstName: String,
  onFirstNameChange: (String) -> Unit,
  email: String,
  onEmailChange: (String) -> Unit,
  password: String,
  onPasswordChange: (String) -> Unit,
  isPasswordVisible: Boolean,
  onPasswordVisibilityToggle: () -> Unit,
  showFirstNameError: Boolean,
  showEmailError: Boolean,
  showPasswordError: Boolean
) {
  val focusManager = LocalFocusManager.current
  val keyboardController = LocalSoftwareKeyboardController.current

  ElevatedCard(
    modifier = Modifier
      .fillMaxWidth()
      .animateContentSize(),
```

```
    shape = CardShape,
    colors = CardDefaults.elevatedCardColors(
      containerColor = MaterialTheme.colorScheme.surface
    )
  ) {
    Column(
      modifier = Modifier.padding(ScreenPadding),
      verticalArrangement = Arrangement.spacedBy(ItemSpacing)
    ) {
      OutlinedTextField(
        value = firstName,
        onValueChange = onFirstNameChange,
        label = { Text("Nombre") },
        placeholder = { Text("Tu nombre") },
        leadingIcon = { Icon(Icons.Default.Person, contentDescription = null) },
        modifier = Modifier.fillMaxWidth(),
        shape = FieldShape,
        singleLine = true,
        keyboardOptions = KeyboardOptions(
          capitalization = KeyboardCapitalization.Words,
          keyboardType = KeyboardType.Text,
          imeAction = ImeAction.Next
        ),
        keyboardActions = KeyboardActions(
          onNext = { focusManager.moveFocus(FocusDirection.Down) }
        ),
        isError = showFirstNameError,
        supportingText = {
          AnimatedVisibility(
            visible = showFirstNameError,
            enter = fadeIn(tween(220)),
            exit = fadeOut(tween(180))
          ) {
            Text("Mínimo 2 caracteres")
          }
        }
      )
```

```
        OutlinedTextField(
          value = email,
          onValueChange = onEmailChange,
          label = { Text("Correo Electrónico") },
          placeholder = { Text("ejemplo@uam.edu.ni") },
          leadingIcon = { Icon(Icons.Default.Email, contentDescription = null) },
          modifier = Modifier.fillMaxWidth(),
          shape = FieldShape,
          singleLine = true,
          keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Email,
            imeAction = ImeAction.Next
          ),
          keyboardActions = KeyboardActions(
            onNext = { focusManager.moveFocus(FocusDirection.Down) }
          ),
          isError = showEmailError,
          supportingText = {
            AnimatedVisibility(
              visible = showEmailError,
              enter = fadeIn(tween(220)),
              exit = fadeOut(tween(180))
            ) {
              Text("Ingresa un correo válido")
            }
          }
        )

        OutlinedTextField(
          value = password,
          onValueChange = onPasswordChange,
          label = { Text("Contraseña") },
          leadingIcon = {
            Crossfade(targetState = isPasswordVisible, label = "passwordLock") {
visible ->
              Icon(
                imageVector = if (visible) Icons.Default.LockOpen else
Icons.Default.Lock,
```

```
            contentDescription = if (visible) {
                "Contraseña visible"
            } else {
                "Contraseña oculta"
            }
        )
    }
},
visualTransformation = if (isPasswordVisible) {
    VisualTransformation.None
} else {
    PasswordVisualTransformation()
},
trailingIcon = {
    IconButton(onClick = onPasswordVisibilityToggle) {
        Icon(
            imageVector = if (isPasswordVisible) {
                Icons.Default.VisibilityOff
            } else {
                Icons.Default.Visibility
            },
            contentDescription = if (isPasswordVisible) {
                "Ocultar contraseña"
            } else {
                "Mostrar contraseña"
            }
        )
    }
},
modifier = Modifier.fillMaxWidth(),
shape = FieldShape,
singleLine = true,
keyboardOptions = KeyboardOptions(
    keyboardType = KeyboardType.Password,
    imeAction = ImeAction.Done
),
keyboardActions = KeyboardActions(
    onDone = {
```

```
                keyboardController?.hide()
                focusManager.clearFocus()
            }
        ),
        isError = showPasswordError,
        supportingText = {
            AnimatedVisibility(
                visible = showPasswordError,
                enter = fadeIn(tween(220)) + slideInVertically(tween(220)) { -it / 2 },
                exit = fadeOut(tween(180)) + slideOutVertically(tween(180)) { -it / 2 }
            ) {
                Text("Mínimo 6 caracteres")
            }
        }
    )
  }
}
}

@Composable
fun LoginFooter(
  canLogin: Boolean,
  isLoading: Boolean,
  onLoginClick: () -> Unit
) {
  val loginEnabled = canLogin && !isLoading
  val buttonBorderColor by animateColorAsState(
    targetValue = if (loginEnabled) {
      MaterialTheme.colorScheme.primary.copy(alpha = 0.35f)
    } else {
      MaterialTheme.colorScheme.outlineVariant
    },
    animationSpec = tween(280),
    label = "buttonBorder"
  )

  var buttonContentWidthPx by remember { mutableIntStateOf(0) }
  val planeOffsetX = remember {
```

```
androidx.compose.animation.core.Animatable(0f) }

  LaunchedEffect(isLoading, buttonContentWidthPx) {
    if (buttonContentWidthPx <= 0) return@LaunchedEffect

    if (isLoading) {
      val travelDistance = (buttonContentWidthPx - 40).coerceAtLeast(0)
      planeOffsetX.snapTo(0f)
      planeOffsetX.animateTo(
        targetValue = travelDistance.toFloat(),
        animationSpec = tween(durationMillis = 900, easing =
FastOutSlowInEasing)
      )
    } else {
      planeOffsetX.snapTo(0f)
    }
  }

  Column(
    modifier = Modifier.fillMaxWidth(),
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.spacedBy(ItemSpacing)
  ) {
    Button(
      onClick = onLoginClick,
      enabled = loginEnabled,
      modifier = Modifier
        .fillMaxWidth()
        .height(56.dp),
      shape = FieldShape,
      border = BorderStroke(1.dp, buttonBorderColor),
      elevation = ButtonDefaults.buttonElevation(defaultElevation = 4.dp)
    ) {
      Box(
        modifier = Modifier
          .fillMaxWidth()
          .onSizeChanged { buttonContentWidthPx = it.width },
        contentAlignment = Alignment.Center
```

```
        ) {
            if (isLoading) {
                Text(
                    text = "Validando...",
                    fontSize = 16.sp,
                    fontWeight = FontWeight.SemiBold
                )
                Icon(
                    imageVector = Icons.AutoMirrored.Filled.Send,
                    contentDescription = "Enviando",
                    modifier = Modifier
                        .align(Alignment.CenterStart)
                        .graphicsLayer { translationX = planeOffsetX.value }
                )
            } else {
                Row(
                    verticalAlignment = Alignment.CenterVertically,
                    horizontalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    Icon(
                        imageVector = Icons.AutoMirrored.Filled.Send,
                        contentDescription = null
                    )
                    Text(
                        text = "Iniciar Sesión",
                        fontSize = 16.sp,
                        fontWeight = FontWeight.SemiBold
                    )
                }
            }
        }
    }

    AnimatedVisibility(visible = !canLogin && !isLoading, enter = fadeIn(), exit =
fadeOut()) {
        Text(
            text = "Completa nombre, correo válido y contraseña de 6+ caracteres",
            style = MaterialTheme.typography.bodySmall,
```

```
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }

    TextButton(onClick = { /* Recuperar pass */ }) {
        Text(
            text = "¿Olvidaste tu contraseña?",
            color = MaterialTheme.colorScheme.secondary
        )
    }
  }
}
```